Низкоуровневый эквивалент построен и согласован. С чего начинать разработку компилятора с Ассемблера?

Чтобы ответить на вопрос, поставленный в заголовке текста, сделаем два действия:

1) Посмотрим на формулировку персонального задания, которая одинакова для всех вариантов и размещена в начале перечня всех вариантов персональных заданий, содержащегося в файле variantn.rtf внутри архива с исходными данными для КР, переданного в группы в начале семестра. Вот она:

«**Необходимо выполнить доработку элементов макета учебной системы программирования до уровня, позволяющего обрабатывать «новые» для макета конструкции языка высокого уровня, примененные в соответствующем варианте**».

2) Для того, чтобы полнее раскрыть смысл лаконичной формулировки персонального задания, приведенной выше, вспомним про учебное пособие [1] (далее в тексте «**Пособие**»), доступного в библиотеке СПбПУ и также переданного в группы внутри архива с исходными данными для КР, раскроем его, найдем в оглавлении **Пособия** раздел «**Рекомендации по порядку выполнения работы**» и прочитаем следующее:

« **. . .**

Данная лабораторная работа предполагает использование в качестве начальной основы отлаженной и работающей программы-макета компилятора с АССЕМБЛЕРА IBM 370, с ограниченным набором языковых конструкций, достаточным для написания законченной правильной программы контрольного примера. Программа-макет компилятора с АССЕМБЛЕРА написана на алгоритмическом языке Си.

Студенту предлагается расширить первичный набор языковых конструкций контрольного примера до уровня, позволяющего оттранслировать без ошибок пример, содержащийся в его личном задании. При этом необходимо:

***а)** Ознакомиться с исходными текстами программы-макета и контрольного примера, на который настроена программа-макет компилятора с АССЕМБЛЕРА,*

***б)** Проверить работоспособность программы-макета компилятора с АССЕМБЛЕРА путем проведения компиляции контрольного примера,*

***в)** Вручную построить ассемблеровский эквивалент исходного текста примера, содержащегося в личном задании, и выделить из него множество языковых конструкций, отсутствующих в ассемблеровском эквиваленте исходного текста контрольного примера,*

***г)** Спроектировать перечень модификаций элементов БД и подпрограмм программы-макета компилятора с АССЕМБЛЕРА с целью обеспечения компиляции ассемблеровского эквивалента примера из личного задания,*

***д)** Ввести модификации в исходный текст программы-макета компилятора с АССЕМБЛЕРА, снабдив измененные места исходного текста построчным комментарием, и записать модифицированный вариант в свой личный директорий на сервере,*

е) *Выполнить отладку модифицированной программы-макета компилятора с АССЕМБЛЕРА,*

ж) *Выполнить отработку своего ассемблеровского эквивалента личного задания с помощью модифицированной программы-макета компилятора с АССЕМБЛЕРА,*

з) *Составить отчет по результатам работы.*

. . .»

Переходим к выполнению рекомендаций из пп. от **а)** до **з).**

Действия по пункту а)

В архиве с исходными данными для КР находим папку:

StudLabRab_2020_magistry\LinuxLabRab\LinuxLabRab.TGZ\LinuxLabRab.tar\.\linux_lab_rab

Убеждаемся, во-первых, в том, что в этой папке имеются С-файлы: **Komppl.c** и **Kompassr.c.** Делая беглый входной контроль содержимого файлов: **Komppl.c** и **Kompassr.c**, убеждается в том, что они и есть исходные тексты для, соответственно, С-программы макета компилятора с ЯВУ и С-программы с языка Ассемблера, которык должны быть доработаны исполнителями КР под свой персональный вариант задания. Далее находим PL/1-файл **examppl.pli** контрольного или, иначе, демо-примера и знакомимся с ним:

```
EXAMP1: PROC OPTIONS ( MAIN );

      DCL A BIN FIXED ( 31 ) INIT ( 111B );
      DCL B BIN FIXED ( 31 ) INIT ( 100B );
      DCL C BIN FIXED ( 31 ) INIT ( 101B );
      DCL D BIN FIXED ( 31 );

      D = A + B - C;

END EXAMP1;
```

Рис.1

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту б)

Устанавливаем на компьютере ОС Linux для 32-хразрядной IBM PC. Переписываем туда архив с исходными данными для КР, включая папку **linux_lab_rab** .

Выполняем те две части shell-скрипта **GenSysProg** (генерация системы программирования), которые относятся к построению, соответственно, из исходного С-файла **Komppl.c** исполняемого файла **Komppl.exe**, а из исходного файла **Kompassr.c** исполняемого файла **Kompassr.exe**/

Выполняем те две части shell-скрипта **StartTestTask**, которые относятся к запуску компилятора с ЯВУ (т.е. файла **Komppl.exe)** для компиляции демо-примера (т.е. файла **exappl.pli**) с получением эквивалента на Ассемблере **examppl.ass** и, далее, объектного эквивалента **examppl.tex**

Убеждаемся в том, что эквивалент демо-примера на Ассемблере **examppl.ass** построен компилятором успешно и что он содержит следующий текст на Ассемблере:

Метка	**КОП**	**Операнды**	**Пояснения**
EXAMP1	START	0	счетчик отн. адреса в нуль
	BALR	RBASE,0	отн. адр. базы в RBASE
	USING	*,RBASE	Объявл. RBASE регистром базы
	L	RRAB,A	Загрузка A в регистр RRAB
	A	RRAB,B	A+B грузим в RRAB
	S	RRAB,C	A+B-C грузим в RRAB
	ST	RRAB,D	A+B-C из RRAB грузим в D
	BCR	15,RVIX	Переход по адр. в рег. RVIX
A	DC	F’3’	A=3
B	DC	F’4’	B=4
C	DC	F’5’	C=5
D	DS	F	Резерв памяти для D
RBASE	EQU	5	RBASE назначим 5
RRAB	EQU	3	RRAB назначим 3
RVIX	EQU	14	RVIX назначим 14
	END		Конец текста блока

Рис.2

Убеждаемся в том, что объектный эквивалент **examppl.tex** построен и содержит следующий набор объектных карт:

*****

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x00**	**2**	**05 50**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x02**	**4**	**58 30 50 12**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x06**	**4**	**5A 30 50 16**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x0A**	**4**	**5B 30 50 1A**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x0E**	**4**	**50 30 50 1E**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x12**	**2**	**07 FE**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x14**	**4**	**00 00 00 03**

	Тип карты		**адр**		**дл**		**Фрагмент двоичного кода**	
	‘TXT’		**0x18**		**4**		**00 00 00 04**	

	Тип карты		**адр**		**дл**		**Фрагмент двоичного кода**	
	‘TXT’		**0x1C**		**4**		**00 00 00 05**	

	Тип карты		**адр**		**дл**		**Фрагмент двоичного кода**	
	‘TXT’		**0x20**		**4**		**XX XX XX XX**	

	Тип карты		
	‘END’		

********** **Конец пакета объектных карт** **********

Рис. 3

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту в)

Продолжение настоящих рекомендаций основано на специально построенном для этой цели условном варианте персонального задания, приведенном ниже и имеющем в своей основе демо-пример, приведенный выше (см. рис. 1):

Условный персональный вариант

```
EXAMP1: PROC OPTIONS ( MAIN );

      DCL A BIN FIXED ( 31 ) INIT ( 111B );
      DCL B BIN FIXED ( 31 ) INIT ( 100B );
      DCL C BIN FIXED ( 31 ) INIT ( 101B );
      DCL D BIN FIXED ( 31 );
      DCL E BIN FIXED ( 31 );
```

```
      D = A + B - C;
      E = D * A / B;

END EXAMP1;
```

Рис. 4

При визуальном сравнении исходных текстов условного персонального задания (см. рис. 4) и демо-примера (см. рис. 1) отмечаем то, что отличия в них минимальны и состоят в расширении:

- терминального алфавита грамматики ЯВУ PL/1 путем введения в него новых терминальных символов «*****» и «/»,
- перечня допустимых арифметических операций, применяемых в арифметическом выражении, находящемся в правой части оператора присваивания, операцией **умножения «*»** и операцией **деления «/»**.

Понятно, что отличия, обнаруженные в исходных текстах условного персонального задания и демо-примера, обусловят отличия их эквивалентов на языке Ассемблер. На рис. 5 приведен один из возможных вариантов такого эквивалента для условного персонального задания, приведенного на рис. 4.

Метка	**КОП**	**Операнды**	**Пояснения**
EXAMP1	START	0	счетчик отн. адреса в нуль
	BALR	RBASE,0	отн. адр. базы в RBASE
	USING	*,RBASE	Объявл. RBASE регистром базы
	L	RRAB,A	Загрузка A в регистр RRAB
	A	RRAB,B	A+B грузим в RRAB
	S	RRAB,C	A+B-C грузим в RRAB
	ST	RRAB,D	A+B-C из RRAB грузим в D
	L	RRAB1+1,D	Загрузка D в регистр RRAB1+1
	M	RRAB1,A	D*A грузим в RRAB1 и RRAB1+1
	D	RRAB1,B	D*A/B→RRAB1+1
	ST	RRAB1+1,E	D*A/B из RRAB1+1 грузим в E
	BCR	15,RVIX	Переход по адр. в рег. RVIX
A	DC	F’3’	A=3
B	DC	F’4’	B=4
C	DC	F’5’	C=5
D	DS	F	Резерв памяти для D
E	DS	F	Резерв памяти для E
RBASE	EQU	5	RBASE назначим 5
RRAB	EQU	3	RRAB назначим 3
RVIX	EQU	14	RVIX назначим 14
RRAB1	EQU	6	RRAB1 назначим 6

	END		Конец текста блока

Рис. 5

Для определения объема приращения функционала макетного варианта компилятора с Ассемблера, исходно рассчитанного только для обработки эквивалента демо-примера (см. рис. 2), с целью обеспечения возможности компиляции эквивалента на Ассемблере для условного персонального задания (см. рис. 5) нужно сравнить эти тексты и выделить строки с «новым» синтаксисом. Сделав это, убедимся, что к таким строкам следует отнести строки ассемблеровского эквивалента персонального задания с операциями умножения (**M**) и деления (**D**), которые имеют следующий синтаксис:

Метка	**КОП**	**Операнды**	**Пояснения**
	M	RRAB1,A	
	D	RRAB1,B	

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

<u>Действия по пункту</u> г)

В основу функционирования макета компилятора с Ассемблера положен (см. **Пособие**) алгоритм, использующий в своей работе, во-первых, БД, включающую в себя следующие элементы:

- исходный текст,
- счетчик адреса (**Сч.Адр**.),
- таблица символов (**TSYM**),
- таблица псевдоопераций (**TPOP**),
- таблица машинных операций (**TMOP**),
- таблица базовых адресов (**TBASE**),
- двоичный эквивалент исходного текста,

и, во-вторых, набор специализированных функциональных блоков, которые могут быть разделены на две группы:

- функциональные блоки обработки ассемблеровских операторов, соответствующих машинным командам. В макетном варианте компилятора с Ассемблера имеется только два блока: для машинных команд **RR** и **RX** типа,
- функциональные блоки обработки ассемблеровских псевдооператоров, к числу которых в макетном варианте компилятора с Ассемблера следует отнести блоки обработки для псевдоопераций: **START**, **DC**, **DS**, **USING**, **EQU**, **END**.

Такая структуризация алгоритма макетного компилятора с Ассемблера облегчает поиск точек модификации для наращивания его функционала в интересах своего варианта задания.
Так, например, как установлено выше, для условного персонального задания необходимо «научить» макетный компилятор обрабатывать ассемблеровские операторы **M** и **D**, соответствующие машинным командам **умножения** и **деления**, обе из которых имеют тип **RX**. Для этого потребуется всего лишь дополнить таблицу машинных операций **TMOP**

двумя строками с характеристиками команд: **M** и **D**. Доработка новых функциональных блоков не потребуется.

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту *д)*

Методика предварительной подготовки и ввода коррекций в исходный текст программы-макета **Kompassr.c** описана в **Пособие.**

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту *е) и ж)*

Отладка программы макета компилятора с Ассемблера, модифицированного правками, производится студентами с опорой на собственный опыт написания и отладки Linux-приложений.
Отладку можно считать законченной в случае, если в процессе экспериментальной проверки работоспособности после запуска отлаживаемой программы компилятора с Ассемблера на выходе генерируется объектный эквивалент входного персонального задания, который совпадает с эталонным вариантом, согласованным с преподавателем на этапе постановки задачи (в ТЗ). В случае условного персонального варианта задания вход соответствует рис. 5, а выход рис. 6.
Работа отлаженной версии компилятора предъявляется (демонстрируется) преподавателю с одновременным вручением отчета для проверки/принятия.

*****

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x06**	**4**	**5A 30 50 26**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x0A**	**4**	**5B 30 50 2A**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x0E**	**4**	**50 30 50 2E**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x12**	**4**	**58 70 50 2E**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x16**	**4**	**5C 60 50 22**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x1A**	**4**	**5D 60 50 24**

Тип карты	**адр**	**дл**	**Фрагмент двоичного кода**
‘TXT’	**0x1E**	**4**	**50 70 50 32**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x22**	**2**	**07FE**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x24**	**4**	**00 00 00 03**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x28**	**4**	**00 00 00 04**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x2C**	**4**	**00 00 00 05**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x30**	**4**	**XX XX XX XX**

Тип карты	адр	дл	Фрагмент двоичного кода
‘TXT’	**0x34**	**4**	**XX XX XX XX**

Тип карты	
‘END’	

************************ Конец пакета объектных карт **************************

Рис. 6

В крайнем случае, если появились проблемы и/или вопросы, то идем с ними за как в консультацией к преподавателю.

<u>Действия по пункту</u> з)

Отчет по результатам составляется с учетом «**Требований к содержанию отчета**», приведенных, как в разделе 4 **Пособия**, так и ниже:
« **. . .**

- описание примера из личного задания, включая его и пояснения модификаций состава и (или) структуры ассемблеровский эквивалент (указать формулировку личного задания),
- постановка задачи модификации программы-макета компилятора с АССЕМБЛЕРА (описание выбранного способа модификации программы-макета),
- перечень БД программы-макета компилятора с АССЕМБЛЕРА,
- перечень и пояснения модификаций алгоритма программы-макета компилятора с АССЕМБЛЕРА (допустимыми являются описания в виде: блок-схемы, текста на Си, а в очевидных случаях обычным текстом),
- выводы по результатам работы (указать позитивные и негативные моменты, проявившиеся в процессе работы, с общей личной оценкой проделанной работы).

. . . »

Если появились проблемы и/или вопросы, то идем с ними за как в консультацией к преподавателю.

Литература

1. Расторгуев В.Я. Разработка элементов учебной системы программирования. Компилятор с Ассемблера [электронный ресурс: учебное пособие], СПбПУ, Спб., 2012